



Structures de Données



Pr. A. MARZAK

2018-2019



1. Structures de données linéaires

Parmi les structures de données linéaires il y a :

les tableaux,

•

les listes chaînées,

les piles,

les files

Les tableaux

Vous connaissez déjà la structure linéaire de type **tableau** pour lequel les éléments de même type le

composant sont placés de façon contigüe en mémoire.

Pour créer un tableau, à 1 ou 2 dimensions, il faut connaître sa taille qui ne pourra être modifiée au

cours du programme, et lui associer un indice pour parcourir ses éléments. Pour les tableaux la

séquence correspond aux numéros des cases du tableau. On accède à un élément du tableau

directement grâce à son indice.

Soit le tableau à 1 dimension suivant nommé Tablo : 12 14 10 24

Pour atteindre la troisième case du tableau il suffit d'écrire Tablo[3] qui contient 10, si les valeurs

de l'indice commencent à 1.

Les tableaux

La structure de type tableau pose des problèmes pour insérer ou supprimer un élément car ces actions nécessitent des décalages du contenu des cases du tableau qui prennent du temps dans l'exécution d'un programme.

Ce type de stockage de valeurs peut donc être coûteux en temps d'exécution. Il existe une autre structure, appelée liste chaînée, pour stocker des valeurs, cette structure permet plus aisément d'insérer et de supprimer des valeurs dans une liste linéaire d'éléments.

Structures de données ?

Définition : une **structure de donnée** est un moyen de coder et de structurer les données manipulées.

Exemple :

type **suite** = tableau [1..N] d'entiers

etudiant = structure

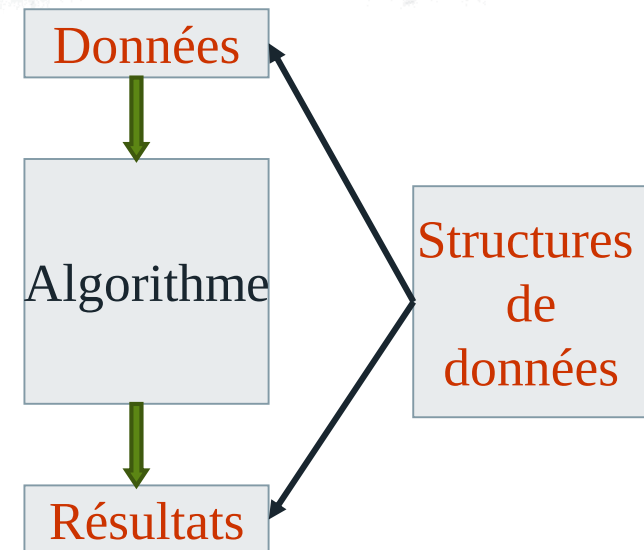
nom, prénom : chaîne de caractères

âge : entier positif

sexe : caractère

...

fin structure



Programme = Algorithme + Structures de données



Structures linéaires. Pile

- **Piles, Files, Listes** font partie des structures dynamiques linéaires.
- Les opérations sur un ensemble dynamique peuvent être regroupées en deux catégories :
 - les requêtes (consultation);
 - les opérations de modification.

Les opérations classiques : $\text{rechercher}(S, \text{clé})$, $\text{Insertion}(S, \text{elt})$, $\text{Supression}(S, \text{elt})$, $\text{Min}(S)$, $\text{Max}(S)$, $\text{Successeur}(S, \text{elt})$, $\text{Prédécesseur}(S, \text{elt})$, etc.

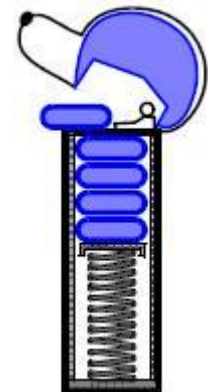


Les piles



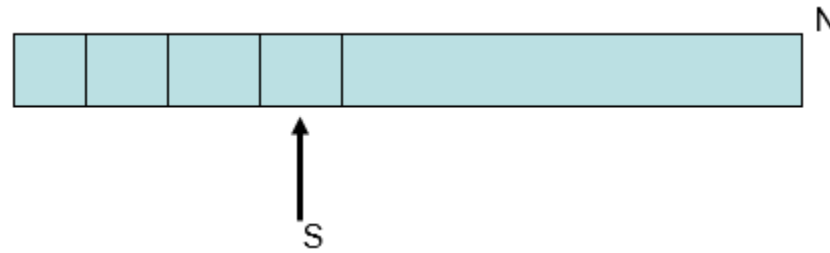
Piles

- Une pile contient des objets insérés et retirés selon le principe du dernier arrivé, premier sorti (**last-in-first-out LIFO**)
- Les objets peuvent être insérés à tout moment, mais seulement le **dernier (le plus récemment inséré)** peut être retiré
- Insérer un objet dans la pile correspond à l'empiler (**pushing**).
Dépiler la pile (**popping**) correspond au retrait d'un objet .
- Analogie: distributeur de bonbons PEZ[®]



Pile

- Pile est un ensemble dynamique dans lequel l'élément supprimé est celui le plus récemment inséré : dernier entré-premier sorti LIFO
- Opérations :
 - Insérer = Empliler
 - Supprimer = Dépiler
 - Valeur = Recherche
- (toujours l'élément pointé par le sommet)
- Illustration graphique dans le cas d'implémentation par un tableau



TDA Pile

- **Type** Pile
- **utilise** Booléen, Élément
- **Opérations**

Pile-vide $PV(P)$	: $\rightarrow$ Pile	{création d'une pile vide
Empiler	: Pile x Element $\rightarrow$ Pile	{EP(P,a)}
Dépiler	: Pile $\rightarrow$ Pile	{supprimer le dernier DP(P)}
Valeur sans modifier	: Pile $\rightarrow$ Élément	{renvoie l'élément au sommet la pile VP(P)}
Est-vide	: Pile $\rightarrow$ Booléen	{test si la pile est vide EV(P)}

Les opérations Dépiler et Valeur ne sont définies que si la pile n'est pas vide

Dépiler(P) est-défini-ssi $\text{est-vide}(P) = \text{faux}$

Valeur(P) est-défini-ssi $\text{est-vide}(P) = \text{faux}$

- **Axiomes**
- $\text{Dépiler}(\text{Empiler}(P,e)) = P$
- $\text{Valeur}(\text{empiler}(P,e)) = e$
- $\text{Est-vide}(\text{Pile-vide}) = \text{vrai}$
- $\text{Est-vide}(\text{Empiler}(P,e)) = \text{faux}$



Evaluation d'une expression

- A l'aide des opérations et des axiomes du TDA Pile évaluer l'expression suivante :
- $EV(DP(DP(EP(EP(PV,a)),b))))?$

Pile. Implémentation à l'aide d'un tableau(1)

- Type Pile = enregistrement
 sommet : entier;
 elts : tableau [1..lmax] d'Element
 FinEnregistrement
- Procédure Pile-vide(réf P: Pile)
- Début
 P.sommet:=0;
 Fin Pile-vide;
 Procédure Empiler(réf P: Pile, val X : Elément);
 Procédure Dépiler(réf P: Pile);
 Fonction Valeur(val P: Pile) : Elément;
 Fonction Est-vide(val P: Pile) : Booléen;

Pile. Implémentation à l'aide d'un tableau(2)

Procédure Empiler(réf P: Pile, val X : Elément)

Début

Si P.sommet=lmax

Alors Erreur « débordement »

Sinon

P.sommet:=P.sommet+1;

P.elts[P.sommet]:=X;

FinSiFinEmpiler

Remarque : chacune des opérations (Empiler, Dépiler) consomme le temps en $O(1)$

Problème: Expliquer comment implémenter deux piles dans un tableau $A[1..n]$ de telle manière qu'aucune ne déborde à moins que le nombre total d'éléments des deux piles vaille n . Les opérations Empiler et Dépiler devront s'exécuter dans un temps en $O(1)$

Pile Dynamique

Une pile dynamique est une liste à la quel on attache un pointeur sommet. C'est un enregistrement à une seule case : pointeur qui pointe la dernière valeur traitée dans la liste (sommet).

De même que pour les piles statiques nous présentons la déclaration et les sous algorithmes de bases détaillés pour le type statique. Et dans un souci d'uniformisation nous utilisons les mêmes noms mais en ajoutant un D (pour rappeler Dynamique) à la fin de chaque nom pour faire la différence.

Note : Il n'y a pas de pile pleine pour les piles dynamique. La seule possibilité de ne pas pouvoir ajouter un élément c'est d'avoir une mémoire pleine, cas que l'on ne prend pas en considération ici.

Pile Dynamique

Déclaration d'une pile dynamique

Type

Liste = ^Elem

Elem = Enregistrement

Info : Type_C ;

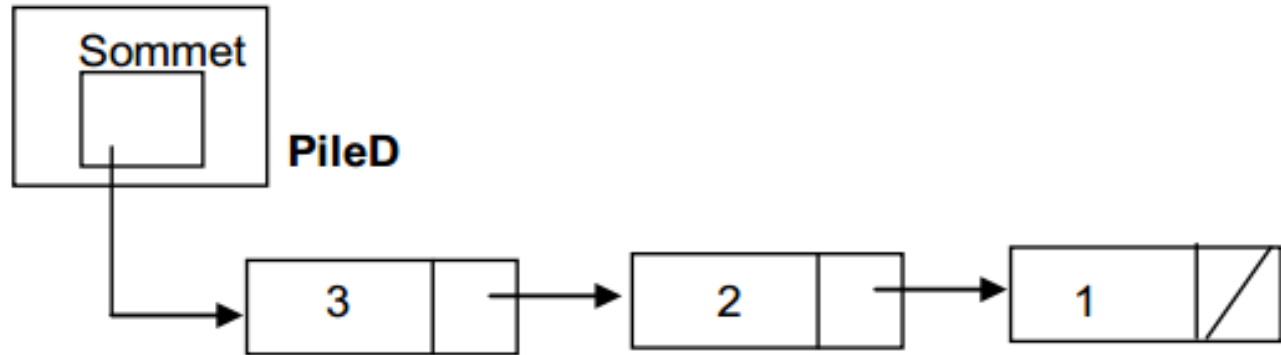
Suiv : ^Elem

Fin ;

PileD = Enregistrement

Sommet : Liste

Fin ;



Note : Les données enregistrées dans la pile peuvent être des entiers, des réels, des caractères, des chaînes de caractères, des booléens, des tableaux, des pointeurs de listes ou encore des piles ou files.

Pile Dynamique

Initialisation d'une pile dynamique

```
Procédure Init_PileD (P : PileD) ;  
Debut  
    P. Sommet ← Nil  
Fin ;
```

Vérification de pile vide dynamique

```
Fonction Pile_videD (P : PileD) : Booleen ;  
Debut  
    Si P. Sommet = Nil Alors  
        Pile_videD ← vrai  
    Sinon  
        Pile_videD ← faux  
    FinSi ;  
Fin ;
```

Une autre façon plus compacte d'écrire cette fonction est la suivante :

```
Fonction Pile_videD (P : PileD) : Booleen ;  
Debut  
    Pile_videD ← P. Sommet = Nil;  
Fin ;
```

Le nom de la fonction recevra le résultat de la comparaison (vrai ou faux) entre le sommet et le zéro.

Pile Dynamique

Ajout d'une nouvelle valeur à une pile dynamique

```
Procédure EmpilerD (Var P : PileD ; X : Type_C) ;  
Variable  
    Pt : Liste ;  
Début  
    Allouer (Pt) ;  
    Pt^.Info ← X ;  
    Pt^.Suiv ← P. Sommet ;  
    P. Sommet ← Pt ;  
Fin ;
```

Note : L'ajout d'une valeur à une pile dynamique revient à une insertion en début de liste si l'on considère que le sommet est la tête de la liste.

Suppression d'une valeur de la pile dynamique

La suppression d'une valeur dans une pile dynamique revient à effectuer une suppression physique d'un nœud (le premier de liste) et à récupérer dans un paramètre, passé par variable, la donnée enregistrée.

```
Procédure DepilerD (Var P : Pile, X : Type_C) ;  
Variable  
    Pt : Liste ;  
Début  
    Si Pile_videD (P) Alors  
        Ecrire("Impossible la pile est vide")  
    Sinon  
        Début  
            Pt ← P. Sommet ;  
            X ← P. Sommet^.Info ;  
            P. Sommet ← P. Sommet^.Suiv ;  
            Libérer (Pt)  
        Fin  
    FinSi ;  
Fin ;
```



Les fichiers

Les fichiers

➤ Motivation

Toutes les structures de données que nous avons vues ont un point commun:

- **Résident dans la mémoire principale** de l'ordinateur. (existence limitée à la durée d'exécution du programme)
- ↳ **Besoin de conserver des données** (pour une utilisation future) après l'exécution du programme (exemple : carnet d'adresses)

☞ la notion de fichier.

Les fichiers

➤ Qu'est-ce qu'un fichier?

Un fichier est une collection de fiches;

Exemple : un fichier dans une bibliothèque, une administration etc.

Trouver une fiche ?

- parcourir le fichier fiche après fiche
- utiliser une clé d'accès, si le fichier est trié selon cette clé (ordre alphabétique, ordre de cotation des livres...).

Les fichiers

➤ En informatique :

Un **fichier** est une séquence de données du même type enregistrées sur un support informatique (de manière **permanente**).

- Un fichier est :
 - conservés en *mémoire secondaire*
(disques et bandes magnétiques, disquettes,...)
 - désigné par un nom et possède des attributs tels que date de création, taille,...

Fichiers : organisation et accès

➤ *Définition*

L'organisation d'un fichier est la manière dont sont rangés **physiquement** les enregistrements du fichier.

- Le but est d'arranger les enregistrements de manière à y accéder le plus rapidement possible.
- L'organisation est choisie à la création du fichier.
- Le choix d'une organisation correspond à un compromis entre rapidité d'accès et espace de stockage disponible.

Fichiers : organisation séquentielle

- Elle ne permet qu'un seul accès : **le séquentiel**.

Toutes les informations sont enregistrées de façon **séquentielle** (linéaire) les unes à la suite des autres.

Pour accéder à une information particulière, il faut nécessairement parcourir le fichier à partir du début, et ce jusqu'au moment où cette information est retrouvée.

Fichiers : organisation relative (accès direct)

- Chaque enregistrement possède un **numéro**.
- On accède à la fiche recherchée en spécifiant ce numéro d'enregistrement.
- L'indication d'un numéro permet donc un **accès direct** à l'information ainsi référencée.

Fichiers : organisation indexée

➤ Notion d 'index :

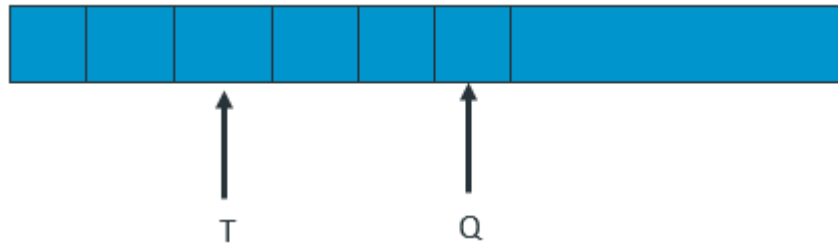
- Soit un fichier F dont les enregistrements E possèdent une clé C (e.g. nom). Un index permet d 'associer à chaque clé C le rang R de l 'enregistrement E de clé C dans F.
- L'index est alors une « [table des matières](#) » du fichiers
- On crée des fichiers supplémentaire d'index
- On parcourt un index pour rechercher une clef. On obtient ainsi l'adresse exacte de l'information recherchée.

Les types de fichiers

- On distingue deux catégories:
 - *Les fichiers binaires* contenant du code binaire représentant chaque élément (enregistrement). Ces fichiers ne doivent être manipulés que par des programmes!
 - *Les fichiers textes* (appelés aussi imprimables) contenant des caractères et susceptibles d'être lus, éditées, imprimés...

TDA File(1)

- Dans le cas d'une file on fait les adjonctions à une extrémité, les accès et les suppressions à l'autre extrémité.
- Les files sont aussi appelées FIFO (first –in –first out) : premier-entré-premier-sorti
- Une file comporte une **tête** et une **queue**. Les éléments sont donc retirés et consultés dans la tête et rajoutés dans la queue



TDA File(2)

- **Type** File
- **utilise** Booléen, Élément
- **Opérations**

File-vide	: $\rightarrow$ File	{création d'une file vide
FV(F)}		
Enfiler	: File x Élément $\rightarrow$ File	{EF(F,a)}
Défiler	: File $\rightarrow$ File	{supprimer le dernier DF(F)}
Valeur	: File $\rightarrow$ Élément	{renvoie l'élément au sommet
sans modifier		la file VF(F)}
Est-vide	: File $\rightarrow$ Booléen	{test si la file est vide EV(F)}

Les opérations Défiler et Valeur ne sont définies que si la file n'est pas vide

Défiler(F) est-défini-ssi $\text{est-vide}(F) = \text{faux}$

Valeur(F) est-défini-ssi $\text{est-vide}(F) = \text{faux}$

- **Axiomes**

$\text{Est-vide}(F) = \text{vrai} \Rightarrow \text{Valeur}(\text{Enfiler}(F, e)) = e$

$\text{Est-vide}(F) = \text{faux} \Rightarrow \text{Valeur}(\text{Enfiler}(F, e)) = \text{Valeur}(F)$

$\text{Est-vide}(F) = \text{vrai} \Rightarrow \text{Défiler}(\text{Enfiler}(F, e)) = \text{File-vide}$

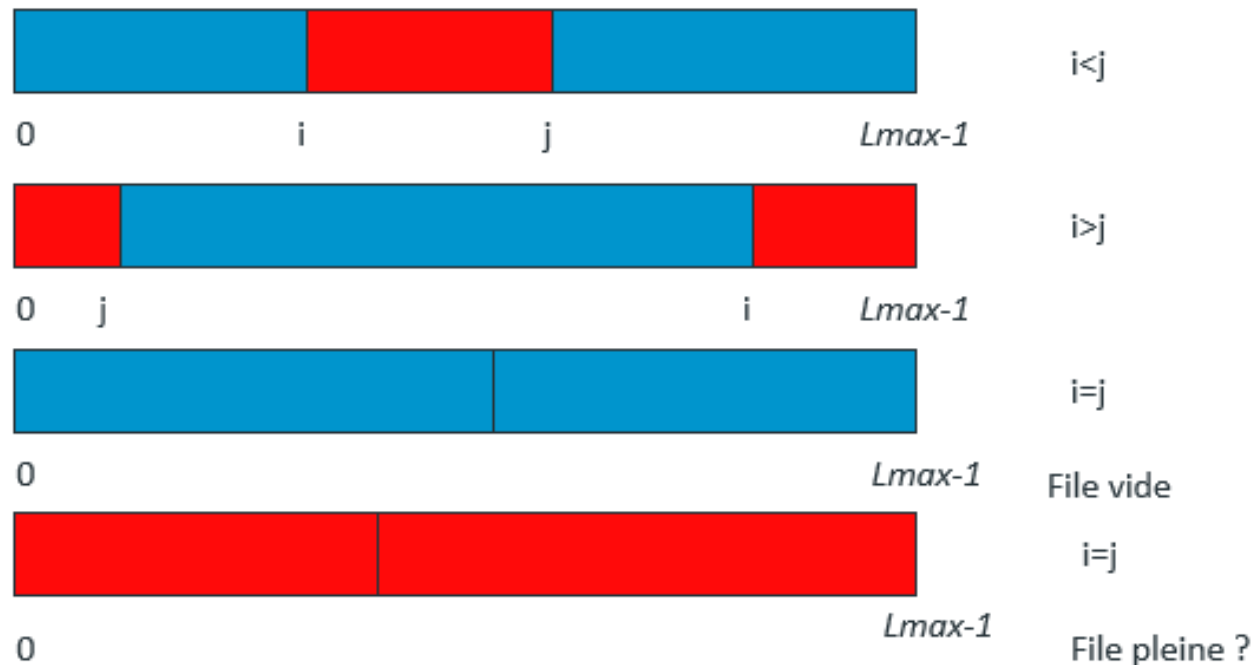
$\text{Est-vide}(F) = \text{faux} \Rightarrow \text{Défiler}(\text{Enfiler}(F, e)) = \text{Enfiler}(\text{Défiler}(F), e)$

$\text{Est-vide}(\text{File-vide}) = \text{vrai}$

$\text{Est-vide}(\text{Enfiler}(F, e)) = \text{faux}$

Représentation contiguë des files

➤ Moyens de représentation : tableau



On fait progresser les indices modulo taille L_{max} du tableau

Représentation contiguë des files(2)

- Gérer les débordements

(1) Enfiler

- $j := j + 1 \bmod l_{\max}$
- Si $j = i$ alors File-pleine
- sinon
- Si File-vide(F) alors $i := j$; FSi
- $\text{Tab}[j] := e$;
- FSi

(2) Défiler

- Si !File-vide(F)
- $i := i + 1 \bmod l_{\max}$
- $e := \text{Tab}[i]$;
- Si $i = j$ alors
- $j := -1$; $i := -1$;
- FSi

(3) File vide $\Leftrightarrow i = -1$ et $j = -1$

File Dynamique

Une File dynamique est une liste à la quel on attache deux (2) pointeurs Debut et Queue. C'est un enregistrement à deux cases : pointeur qui pointe le premier élément de la liste (Debut) et un autre qui pointe la dernière valeur ajoutée dans la liste (Queue).

- Exemples :

1- Une file vide $\rightarrow \text{Debut} = \text{Queue} = \text{Nil}$

2- Une file a un seul élément $\rightarrow \text{Debut} = \text{Queue} \neq \text{Nil}$

3- Une file a plus d'un élément $\rightarrow \text{Debut} \neq \text{Queue}$

De même que pour les files statiques nous présentons la déclaration et les sous algorithmes de bases détaillés pour le type statique. Et dans un souci d'uniformisation nous utilisons les mêmes noms mais en ajoutant un D (pour rappeler Dynamique) à la fin de chaque nom pour faire la différence

Note : Il n'y a pas de file pleine pour les files dynamique. La seule possibilité de ne pas pouvoir ajouter un élément c'est d'avoir une mémoire pleine, cas que l'on ne prend pas en considération ici.

File Dynamique

Déclaration d'une file dynamique

Type

Liste = ^Elem

Elem = Enregistrement

Info : Type_C ;

Suiv : ^Elem

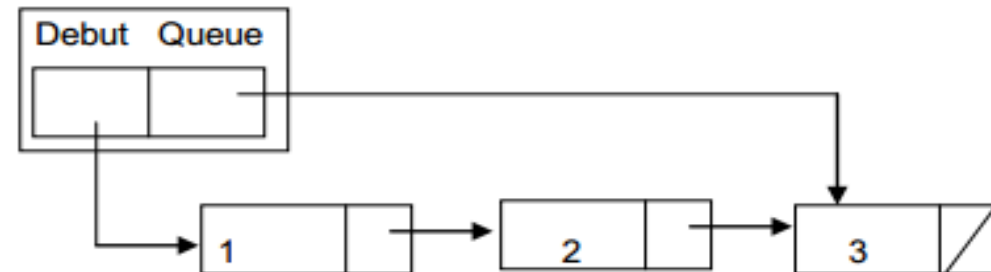
Fin ;

FileD = Enregistrement

Sommet, Queue : Liste

Fin ;

FileD



Note : Les données enregistrées dans la file peuvent être des entiers, des réels, des caractères, des chaînes de caractères, des booléens, des tableaux, des pointeurs de listes ou encore des piles ou files.

Initialisation d'une file dynamique

Procedure Init_FileD (F : FileD) ;

Debut

F. Debut $\leftarrow$ Nil ;

F. Queue $\leftarrow$ Nil

Fin ;

File Dynamique

Vérification de File vide dynamique

```
Fonction File_videD (F : FileD) : Booleen ;  
Debut  
  Si F. Queue = Nil Alors  
    File_videD ← vrai  
  Sinon  
    File_videD ← faux  
  FinSi ;  
Fin ;
```

Une autre façon plus compacte d'écrire cette fonction est la suivante :

```
Fonction File_videD (F : FileD) : Booleen ;  
Debut  
  File_videD ← F. Queue = Nil;  
Fin ;
```

Le nom de la fonction recevra le résultat de la comparaison (vrai ou faux) entre la queue et le Nil.

File Dynamique

Ajout d'une nouvelle valeur à une file dynamique

L'ajout d'une valeur à une File dynamique revient à une insertion à la fin de liste avec l'adresse du dernier élément dans Queue. Le cas d'un enfiler sur file vide nécessite l'initialisation du Debut à l'adresse du nouveau nœud.

```
Procédure EnfilerD (Var F : FileD ; X : Type_C) ;  
Variable  
  Pt : Liste ;  
Debut  
  Allouer (Pt) ;  
  Pt^.Info ← X ;  
  Pt^.Suiv ← Nil ;  
  Si File_videD (F) Alors  
    F. Debut ← Pt  
  Sinon  
    F. Queue^.Suiv ← Pt  
  FinSi ;  
  F. Queue ← Pt ;  
Fin ;
```


File Dynamique

Suppression d'une valeur de la file dynamique

La suppression d'une valeur dans une file dynamique revient à effectuer une suppression physique d'un nœud (le premier de liste) et à récupérer dans un paramètre, passé par variable, la donnée enregistrée.

```
Procedure DefilerD (Var F: FileD, X : Type_C) ;  
Variable  
  Pt : Liste ;  
Debut  
  Si File_videD (F) Alors  
    Écrire('Impossible la file est vide')  
  Sinon  
    Debut  
      Pt ← F. Debut ;  
      X ← F.Debut^.Info ;  
      F. Debut ← F. Debut^.Suiv ;  
      Si F. Debut = Nil Alors  
        F. Queue ← Nil  
      FinSi /* si debut devient Nil cela veut dire que la file a été vidée et Queue doit devenir Nil*/  
      Libérer (Pt)  
    Fin  
  FinSi ;  
Fin ;
```



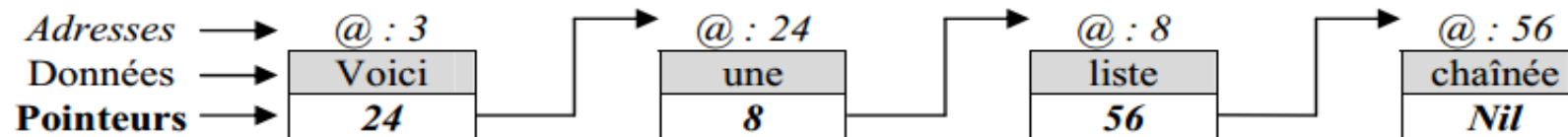

Listes chaînées

Listes chaînées(1)

Une liste chaînée est une structure linéaire qui n'a pas de dimension fixée à sa création. Ses éléments de même type sont éparpillés dans la mémoire et reliés entre eux par des pointeurs. Sa dimension peut être modifiée selon la place disponible en mémoire. La liste est accessible uniquement par sa tête de liste c'est-à-dire son premier élément.

Pour les listes chaînées la séquence est mise en oeuvre par le pointeur porté par chaque élément qui indique l'emplacement de l'élément suivant. Le dernier élément de la liste ne pointe sur rien (Nil). On accède à un élément de la liste en parcourant les éléments grâce à leurs pointeurs.

Soit la liste chaînée suivante (@ indique que le nombre qui le suit représente une adresse) :



Pour accéder au troisième élément de la liste il faut toujours débiter la lecture de la liste par son premier élément dans le pointeur duquel trouve la position du troisième élément... Pour ajouter, supprimer ou déplacer un élément il suffit d'allouer une place en mémoire et de mettre à jour les pointeurs des éléments. est indiqué la position du deuxième élément. Dans le pointeur du deuxième élément de la liste on

Listes chaînées(1)

Il existe différents types de listes chaînées :

- **Liste chaînée simple** constituée d'éléments reliés entre eux par des pointeurs.
- **Liste chaînée ordonnée** où l'élément suivant est plus grand que le précédent. L'insertion et la suppression d'élément se font de façon à ce que la liste reste triée.
- **Liste doublement chaînée** où chaque élément dispose non plus d'un mais de deux pointeurs pointant respectivement sur l'élément précédent et l'élément suivant. Ceci permet de lire la liste dans les deux sens, du premier vers le dernier élément ou inversement.
- **Liste circulaire** où le dernier élément pointe sur le premier élément de la liste. S'il s'agit d'une liste doublement chaînée alors de premier élément pointe également sur le dernier.

Ces différents types peuvent être mixés selon les besoins. On utilise une liste chaînée plutôt qu'un tableau lorsque l'on doit traiter des objets représentés par des suites sur lesquelles on doit effectuer de nombreuses suppressions et de nombreux ajouts. Les manipulations sont alors plus rapides qu'avec des tableaux.

Listes chaînées(2)

Définitions

Un élément d'une liste est l'ensemble (ou structure) formé :

- d'une donnée ou information,
- d'un pointeur nommé Suivant indiquant la position de l'élément le suivant dans la liste. A chaque élément est associée une adresse mémoire.

Les listes chaînées font appel à la notion de variable dynamique. Une variable dynamique:

- est déclarée au début de l'exécution d'un programme,
- elle y est créée, c'est-à-dire qu'on lui alloue un espace à occuper à une adresse de la mémoire,
- elle peut y être détruite, c'est-à-dire que l'espace mémoire qu'elle occupait est libéré,
- l'accès à la valeur se fait à l'aide d'un pointeur.

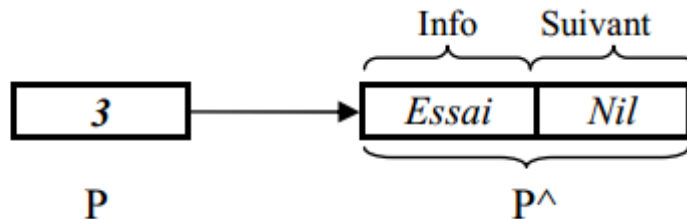
Listes chaînées(2)

Un pointeur est une variable dont la valeur est une adresse mémoire (voir chapitre 9). Un pointeur, noté P , pointe sur une variable dynamique notée $P^\wedge$.

Le type de base est le type de la variable pointée.

Le type du pointeur est l'ensemble des adresses des variables pointées du type de base. Il est représenté par le symbole $^\wedge$ suivi de l'identificateur du type de base.

Exemple:



La variable pointeur P pointe sur l'espace mémoire $P^\wedge$ d'adresse 3. Cette cellule mémoire contient la valeur "Essai" dans le champ Info et la valeur spéciale Nil dans le champ Suivant. Ce champ servira à indiquer quel est l'élément suivant lorsque la cellule fera partie d'une liste. La valeur Nil indique qu'il n'y a pas d'élément suivant. $P^\wedge$ est l'objet dont l'adresse est rangée dans P .

Listes chaînées(2)

Les listes chaînées entraînent l'utilisation de procédures d'allocation et de libération dynamiques de la mémoire. Ces procédures sont les suivantes:

- **Allouer(P)**: réserve un espace mémoire $P^{\wedge}$ et donne pour valeur à P l'adresse de cet espace mémoire. On alloue un espace mémoire pour un élément sur lequel pointe P.
- **Désallouer(P)** : libère l'espace mémoire qui était occupé par l'élément à supprimer $P^{\wedge}$ sur lequel pointe P. Pour définir les variables utilisées dans l'exemple ci-dessus, il faut :
- définir le type des éléments de liste : Type Cellule= Structure

Info : Chaîne

Suivant : Liste

fin Structure

- définir le type du pointeur : Type Liste = $^{\wedge}$ Cellule
 - déclarer une variable pointeur : Var P : Liste
 - allouer une cellule mémoire qui réserve un espace en mémoire et donne à P la valeur de l'adresse de l'espace mémoire $P^{\wedge}$: Allouer(P)
 - affecter des valeur à l'espace mémoire $P^{\wedge}$: $P^{\wedge}.\text{Info} \leftarrow \text{"Essai"} ; P^{\wedge}.\text{Suivant} \leftarrow \text{Nil}$
- Quand P = Nil alors P ne pointe sur rien.

Listes chaînées simples

Une liste chaînée simple est composée :

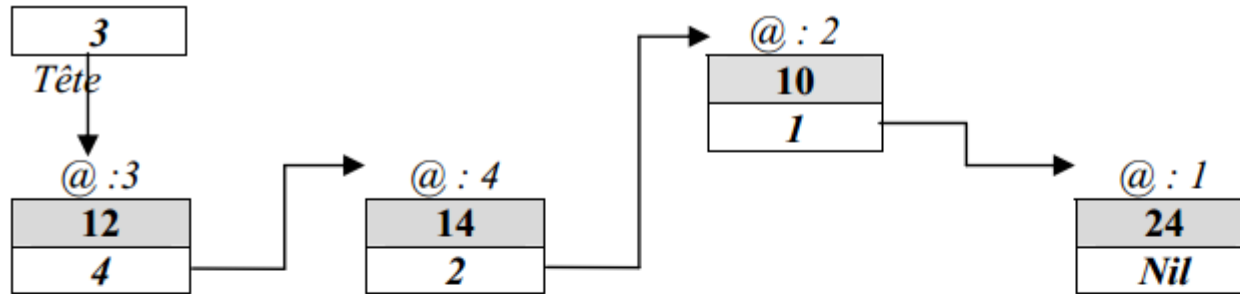
- d'un ensemble d'éléments tel que chacun :
 - o est rangé en mémoire à une certaine adresse,
 - o contient une donnée (Info),
 - o contient un pointeur, souvent nommé Suivant, qui contient l'adresse de l'élément suivant dans la liste,
- d'une variable, appelée Tête, contenant l'adresse du premier élément de la liste chaînée.

Le pointeur du dernier élément contient la valeur Nil. Dans le cas d'une liste vide le pointeur de la tête contient la valeur Nil. Une liste est définie par l'adresse de son premier élément.

Avant d'écrire des algorithmes manipulant une liste chaînée, il est utile de montrer un schéma représentant graphiquement l'organisation des éléments de la liste chaînée.

Listes chaînées simples

Exemple:



Le 1^{er} élément de la liste vaut 12 à l'adresse 3 (début de la liste chaînée)

Le 2^e élément de la liste vaut 14 à l'adresse 4 (car le pointeur de la cellule d'adresse 3 est égal à 4)

Le 3^e élément de la liste vaut 10 à l'adresse 2 (car le pointeur de la cellule d'adresse 4 est égal à 2)

Le 4^e élément de la liste vaut 24 à l'adresse 1 (car le pointeur de la cellule d'adresse 2 est égal à 1)

Si P a pour valeur 3

$P^{\wedge}.\text{Info}$ a pour valeur 12

$P^{\wedge}.\text{Suivant}$ a pour valeur 4

Si P a pour valeur 2

$P^{\wedge}.\text{Info}$ a pour valeur 10

$P^{\wedge}.\text{Suivant}$ a pour valeur 1

Listes chaînées simples

Traitements de base d'utilisation d'une liste chaînée simple

Il faut commencer par définir un type de variable pour chaque élément de la chaîne. En langage algorithmique ceci se fait comme suit :

```
Type Liste = ^Element
Type Element = Structure
    Info : variant
    Suivant : Liste
Fin structure
```

Variables Tete, P : Liste

Le type de *Info* dépend des valeurs contenues dans la liste : entier, chaîne de caractères, variant pour un type quelconque...

Listes chaînées simples

Créer une liste chaînée composée de 2 éléments de type chaîne de caractères. Déclarations des types pour la liste :

Type Liste = ^Element

Type Element = Structure

Info : chaîne de caractères

Suivant : Liste

Fin structure

Algorithme CréationListe2Elements

Tete, P : Liste

NombreElt : entier

DEBUT

1	Tete ← Nil	/* pour l'instant la liste est vide */
2	Allouer(P)	/* réserve un espace mémoire pour le premier élément */
3	Lire(P^.Info)	/* stocke dans l'Info de l'élément pointé par P la valeur saisie */
4	P^.Suivant ← Nil	/* il n'y a pas d'élément suivant */
5	Tete ← P	/* le pointeur Tete pointe maintenant sur P */
	/* Il faut maintenant ajouter le 2 ^e élément, ce qui revient à insérer un élément en tête de liste */	
6	Allouer(P)	/* réserve un espace mémoire pour le second élément */
7	Lire(P^.Info)	/* stocke dans l'Info de l'élément pointé par P la valeur saisie */
8	P^.Suivant ← Tete	/* élément inséré en tête de liste */
9	Tete ← P	

FIN

Listes chaînées simples

Créer une liste chaînée composée de 2 éléments de type chaîne de caractères. Déclarations des types pour la liste :

Type Liste = $\wedge$ Element

Type Element = Structure

Info : chaîne de caractères

Suivant : $\text{Liste}^{\wedge}$

Fin structure

Algorithme CréationListe2Elements

Tete, P : Liste

NombreElt : entier

DEBUT

1	Tete $\leftarrow$ Nil	/* pour l'instant la liste est vide */
2	Allouer(P)	/* réserve un espace mémoire pour le premier élément */
3	Lire(P^.Info)	/* stocke dans l'Info de l'élément pointé par P la valeur saisie */
4	P^.Suivant $\leftarrow$ Nil	/* il n'y a pas d'élément suivant */
5	Tete $\leftarrow$ P	/* le pointeur Tete pointe maintenant sur P */
	/* Il faut maintenant ajouter le 2 ^e élément, ce qui revient à insérer un élément en tête de liste */	
6	Allouer(P)	/* réserve un espace mémoire pour le second élément */
7	Lire(P^.Info)	/* stocke dans l'Info de l'élément pointé par P la valeur saisie */
8	P^.Suivant $\leftarrow$ Tete	/* élément inséré en tête de liste */
9	Tete $\leftarrow$ P	

FIN

Listes chaînées simples

Créer une liste chaînée composée de plusieurs éléments de type chaîne de caractères

Déclarations des types pour la liste :

Type Liste = ^Element

Type Element = Structure

Info : chaîne de caractères

Suivant : Liste

fin Structure

Pour créer une liste chaînée contenant un nombre d'éléments à préciser par l'utilisateur il suffit d'introduire deux variables de type Entier NombreElt et Compteur

- de faire saisir la valeur de NombreElt par l'utilisateur dès le début du programme,
- d'écrire une boucle Pour Compteur allant de 1 à NombreElt comprenant les instructions 6, 7, 8 et 9.

Listes chaînées simples

Algorithme CréationListeNombreConnu

Tete, P : Liste
NombreElt : entier
Compteur : entier

DEBUT

Lire(NombreElt)

Tete $\leftarrow$ Nil

POUR Compteur **DE** 1 **A** NombreElt **FAIRE**

 Allouer(P)

 /* réserve un espace mémoire pour l'élément à ajouter */

 Lire(P^.Info)

 /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

 P^.Suivant $\leftarrow$ Tete

 /* élément inséré en tête de liste */

 Tete $\leftarrow$ P

 /* le pointeur Tete pointe maintenant sur P */

FIN POUR

FIN

Algorithme CréationListeNombreInconnu

Tete, P : Liste

Valeur : chaîne de caractères

DEBUT

Tete $\leftarrow$ Nil

Lire (Valeur)

TANT QUE que Valeur $\neq$ "XXX" **FAIRE**

 Allouer(P)

 /* réserve un espace mémoire pour l'élément à ajouter */

 P^.Info $\leftarrow$ Valeur

 /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

 P^.Suivant $\leftarrow$ Tete

 /* élément inséré en tête de liste */

 Tete $\leftarrow$ P

 /* le pointeur Tete pointe maintenant sur P */

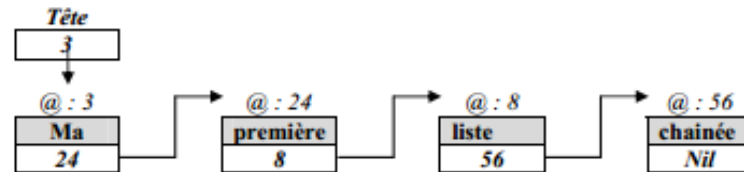
 Lire (Valeur)

FIN TANT QUE

FIN

Listes chaînées simples

Afficher les éléments d'une liste chaînée



L'algorithme est donné sous forme d'une procédure qui reçoit la tête de liste en paramètre.

Procédure AfficherListe (Entrée P : Liste)

/* Afficher les éléments d'une liste chaînée passée en paramètre */

DEBUT

1 P ← Tete

/* P pointe sur le premier élément de la liste */

TANT QUE P <> NIL **FAIRE** /* On parcourt la liste tant que l'adresse de l'élément suivant n'est pas Nil */

2 Ecrire(P^.Info)

/* si la liste est vide Tete est à Nil */

/* afficher la valeur contenue à l'adresse pointée par P */

3 P ← P^.Suivant

/* On passe à l'élément suivant */

FIN TANT QUE

FIN

1	P a pour valeur 3
2	"Ma" s'affiche
3	P prend pour valeur 24
2	"première" s'affiche
3	P prend pour valeur 8
2	"liste" s'affiche
3	P prend pour valeur 56
2	"chainée" s'affiche
3	P prend pour valeur Nil

On s'arrête puisque P a pour valeur Nil et que c'est la condition d'arrêt de la boucle Tant Que.

Listes chaînées simples

Rechercher une valeur donnée dans une liste chaînée ordonnée

Dans cet exemple nous reprenons le cas de la liste chaînée contenant des éléments de type chaîne de caractères, mais ce pourrait être tout autre type, selon celui déterminé à la création de la liste (rappelons que tous les éléments d'une liste chaînée doivent avoir le même type). La liste va être parcourue à partir de son premier élément (celui pointé par le pointeur de tête). Il a deux cas d'arrêt :

- avoir trouvé la valeur de l'élément,
- avoir atteint la fin de la liste.

L'algorithme est donné sous forme d'une procédure qui reçoit la tête de liste en paramètre. La valeur à chercher est lue dans la procédure.

Listes chaînées simples

```
Procédure RechercherValeurListe  (Entrée Tete : Liste, Val : variant)
/* Rechercher si une valeur donnée en paramètre est présente dans la liste passée en paramètre */
Variables locales
P                : Liste      /* pointeur de parcours de la liste */
Trouve           : booléen   /* indicateur de succès de la recherche */

DEBUT
  SI Tete <> Nil ALORS           /* la liste n'est pas vide on peut donc y chercher une valeur */
    P ← Tete
    Trouve ← Faux
    TANTQUE P <> Nil ET Non Trouve
      SI P^.Info = Val ALORS      /* L'élément recherché est l'élément courant */
        Trouve ← Vrai
      SINON                      /* L'élément courant n'est pas l'élément recherché */
        P ← P^.Suivant          /* on passe à l'élément suivant dans la liste */
      FINSI
    FIN TANT QUE
    SI Trouve ALORS
      Ecrire (" La valeur ", Val, " est dans la liste")
    SINON
      Ecrire (" La valeur ", Val, " n'est pas dans la liste")
    FINSI
  SINON
    Ecrire("La liste est vide")
  FINSI
FIN
```

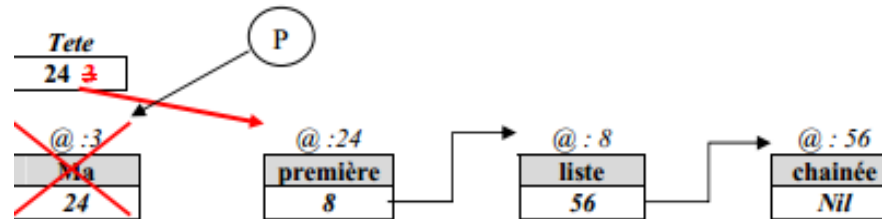
Listes chaînées simples

Supprimer le premier élément d'une liste chaînée

Il y a deux actions, dans cet ordre, à réaliser :

- faire pointer la tête de liste sur le deuxième élément de la liste,
- libérer l'espace mémoire occupé par l'élément supprimé.

Il est nécessaire de déclarer un pointeur local qui va pointer sur l'élément à supprimer, et permettre de libérer l'espace qu'il occupait.



Procédure SupprimerPremierElement (Entrée/Sortie Tete : Liste)

/ Supprime le premier élément de la liste dont le pointeur de tête est passé en paramètre */*

Variables locales

P : Liste

/ pointeur sur l'élément à supprimer */*

DEBUT

SI Tete <> Nil **ALORS**

/ la liste n'est pas vide on peut donc supprimer le premier élément */*

P ← Tete

/ P pointe sur le 1^{er} élément de la liste */*

Tete ← P^.Suivant

/ la tête de liste doit pointer sur le deuxième élément */*

Desallouer(P)

/ libération de l'espace mémoire qu'occupait le premier élément */*

SINON

Ecrire("La liste est vide")

FINSI

FIN

Listes chaînées simples

Supprimer d'une liste chaînée un élément portant une valeur donnée

Il faut:

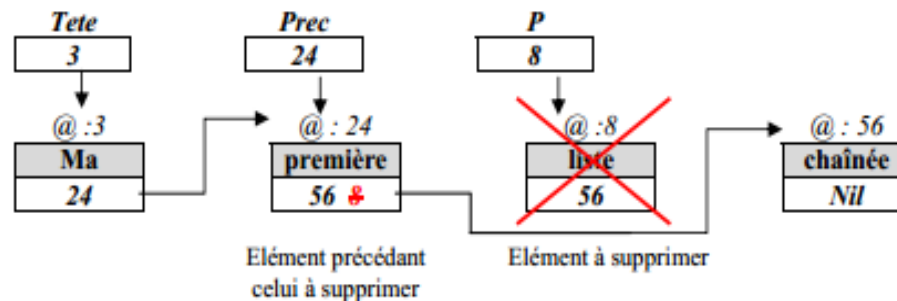
traiter à part la suppression du premier élément car il faut modifier le pointeur de tête,

trouver l'adresse P de l'élément à supprimer,

sauvegarder l'adresse Prec de l'élément précédant l'élément pointé par P pour connaître l'adresse de l'élément précédant l'élément à supprimer, puis faire pointer l'élément précédent sur l'élément suivant l'élément à supprimer,

Libérer l'espace mémoire occupé par l'élément supprimé.

L'exemple considère que l'on souhaite supprimer l'élément contenant la valeur "liste" de la liste ci-dessus.



Listes chaînées simples

Procédure SupprimerElement (Entrée/Sortie Tete : Liste, Val : variant)

/ Supprime l'élément dont la valeur est passée en paramètre */*

Variables locales

P : Liste

/ pointeur sur l'élément à supprimer */*

Prec : Liste

/ pointeur sur l'élément précédant l'élément à supprimer */*

Trouvé : Liste

/ indique si l'élément à supprimer a été trouvé */*

DEBUT

SI Tete <> Nil **ALORS** */* la liste n'est pas vide on peut donc y chercher une valeur à supprimer */*

SI Tete^.info = Val **ALORS** */* l'élément à supprimer est le premier */*

P ← Tete

Tete ← Tete^.Suivant

Desallouer(P)

SINON */* l'élément à supprimer n'est pas le premier */*

Trouve ← Faux

Prec ← Tete

/ pointeur précédent */*

P ← Tete^.Suivant

/ pointeur courant */*

TANTQUE P <> Nil ET Non Trouve

SI P^.info = Val **ALORS** */* L'élément recherché est l'élément courant */*

Trouve ← Vrai

SINON

/ L'élément courant n'est pas l'élément cherché */*

Prec ← P

/ on garde la position du précédent */*

P^ ← P^.Suivant

/ on passe à l'élément suivant dans la liste */*

FINSI

FIN TANT QUE

SI Trouve **ALORS**

Prec^.Suivant ← P^.Suivant

/ on "saute" l'élément à supprimer */*

Desallouer(P)

SINON

Ecrire ("La valeur ", Val, " n'est pas dans la liste")

FINSI

FINSI

SINON

Ecrire ("La liste est vide")

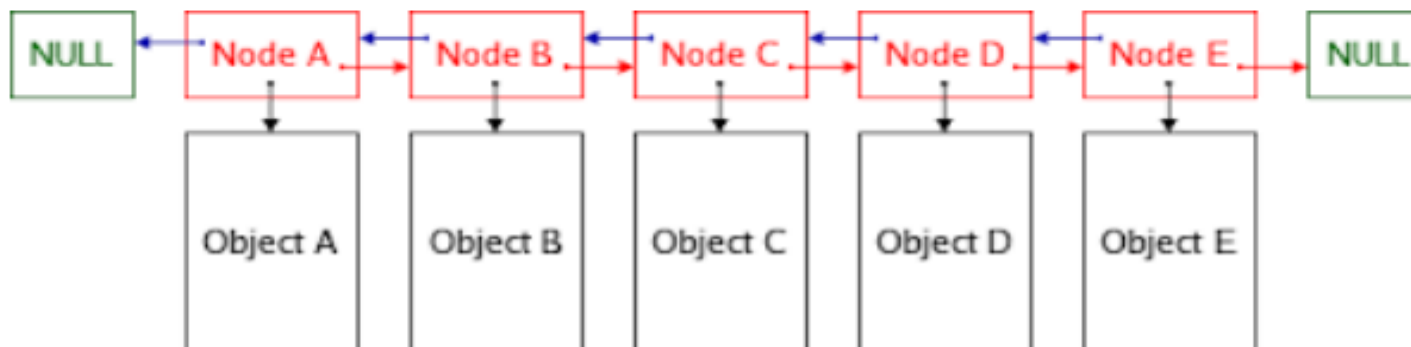
FINSI

FIN

Listes doublement chaînées

Définition :

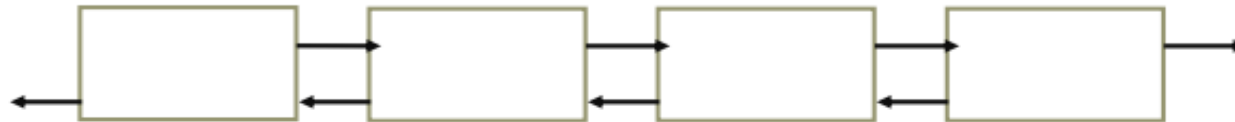
- Une liste doublement chaînée est une structure de données dynamique contenant une séquence finie des cellules.
- La cellule (nœud) est la composante principale d'une liste doublement chaînée



Listes doublement chaînées

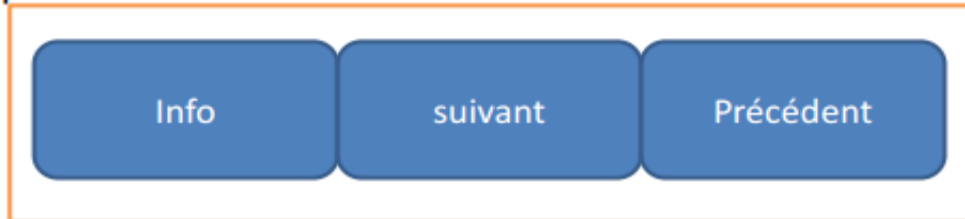
Définition :

Une liste doublement chaînée est une liste chaînée, dans laquelle on peut accéder à l'élément prédécesseur :



Une cellule est une structure qui comporte trois champs :

- **Le champ info** : il contient des informations sur l'élément représenté par la cellule ;
- **Le champ suivant** : il représente un pointeur qui contient l'adresse de la cellule suivante.
- **Le champ précédent** : il représente un pointeur qui contient l'adresse de la cellule précédente



Type Cellule = structure
valeur : entier
Suivant : *Cellule
precedent : *Cellule

Fin

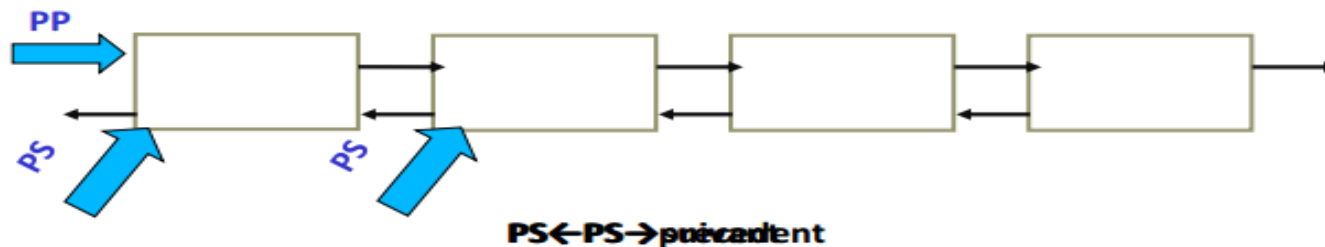
Listes doublement chaînées

Pour une liste doublement chaînée :

- On peut accéder directement au premier et dernier élément
- On peut parcourir la liste dans les 2 sens (on peut donc revenir en arrière).

Manipulation :

Une liste doublement chaînée présente l'avantage de donner accès au maillon précédent : *Avancer* et *reculer*



Remarque : Le pp d'une liste doublement chaînée n'est pas forcément positionné sur la première cellule.

Listes doublement chaînées

Avec un seul pointeur :

Principe :

- 1 cellule / élément
- une liste = pointeur sur le 1er élément
- n = NULL

Implémentation en langage C :

```
typedef struct cellule
{
    int info ; //le champ info peut avoir n'importe quel type
    struct cellule *next ; //pointeur contenant l'adresse de la cellule suivante
    struct cellule *prev ; //pointeur vers la cellule précédente
} DListe ;
```

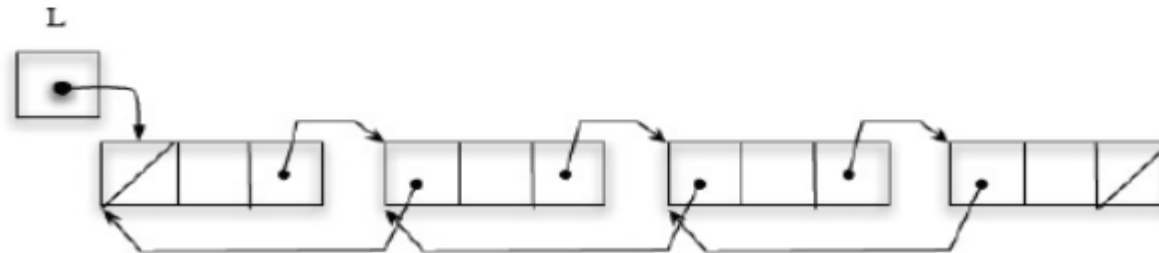

Listes doublement chaînées

Avec un seul pointeur :

Déclaration d'une liste doublement chaînée

- Pour déclarer une variable de type liste doublement chaînée, il suffit de déclarer un pointeur sur le premier élément.
- Exemple : `DListe * L;`

Représentation graphique :



Listes doublement chaînées

Avec deux pointeurs :

Principe

- 1 cellule / élément
- une liste = un pointeur sur le premier élément + un pointeur sur le dernier élément
- fin = NULL

Implémentation en C

```
typedef struct cellule{
    int info ; //le champ info peut avoir n'importe quel type
    struct cellule *next ; //pointeur contenant l'adresse de la cellule suivante
    struct cellule *prev ; //pointeur vers la cellule précédente
}Cellule ;
/* type des listes doublement chaînées*/
typedef struct{
    Cellule *first ; //pointeur contenant l'adresse de la première cellule
    Cellule *last ; //pointeur contenant l'adresse de la dernière cellule
}Dliste ;
```

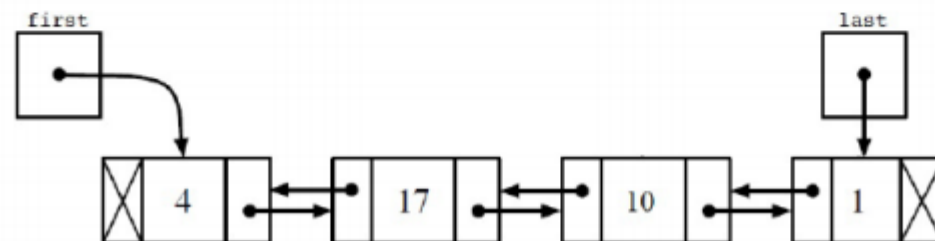
Listes doublement chaînées

Avec deux pointeurs :

Déclaration d'une liste doublement chaînée

- Pour déclarer une variable de type liste doublement chaînée, il existe deux possibilités :
 - En utilisant une variable statique : Dliste L.
 - En utilisant une variable dynamique : Dliste *L.
- La variable L est une structure contenant deux champs : L.First et L.Last

Représentation graphique

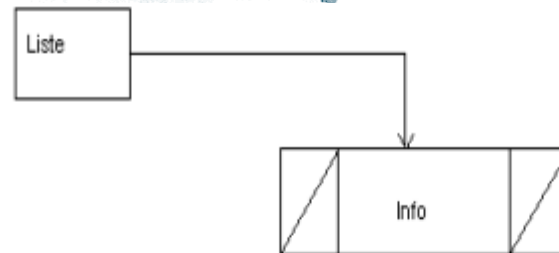


Listes doublement chaînées

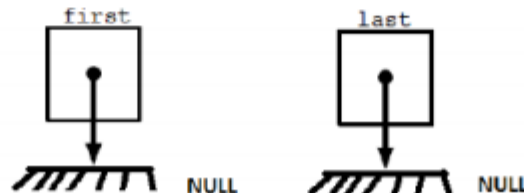
Les principales opérations sur une liste doublement chaînée

- 1. Insertion**
2. Affichage
3. Suppression

Création d'une Liste doublement chaînée vide



```
Dliste newEmptyDlist(void)
{
    Dliste *L;
    L = (Dliste*) malloc(sizeof(Dliste)) ;
    L.first = NULL ;
    L.last = NULL ;
    return(L) ;
}
```



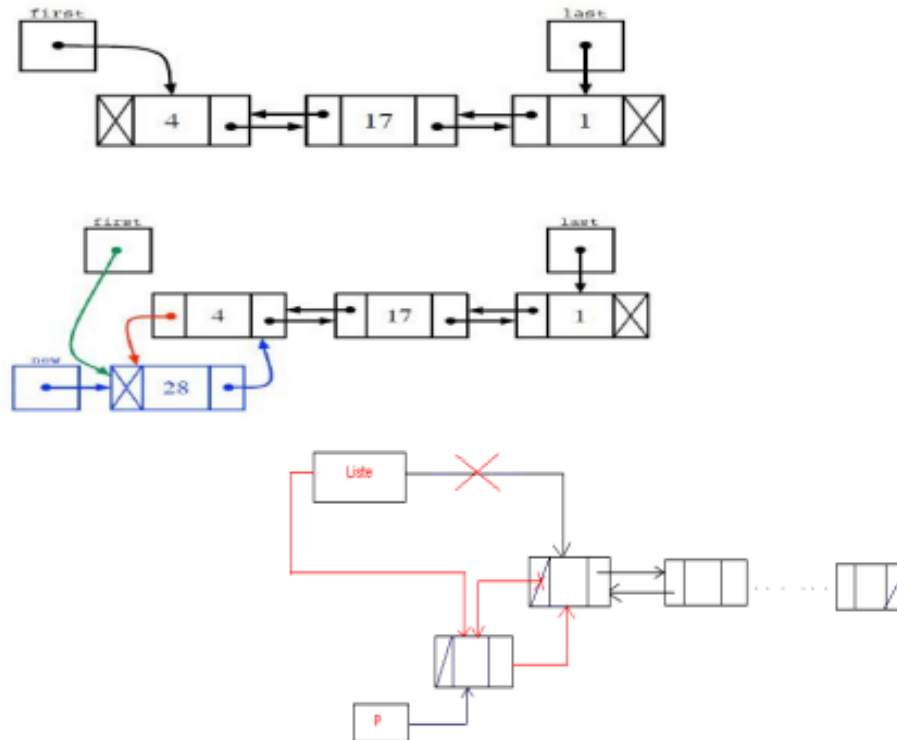
Initialisation d'une Liste doublement chaînée

Opération : **Initialisation** :

```
Void initialisation (Dliste liste, int x)
{
    Dliste * p ;
    P = (liste_db) malloc (sizeof (Dliste)) ;
    p → info = x ;
    p → suiv = null ;
    Liste = p ;
}
```

Listes doublement chaînées

Insertion au début



Listes doublement chaînées

Insertion au début

```
void insertDebut(Dliste *L, int e)
{
    Cellule *nouv;
    nouv = newElement(NULL,L->first,e);
    if(L->first != NULL)
        (L->first)->precedent=nouv;
    else
        L->last=nouv;
    L->first=nouv;
}
```

```
int isEmpty(Dliste l)
{
    if (l.first == NULL)
        return 1 ;
    else
        return 0 ;
}
```

```
Void inser_debut (Dliste *liste, int x)
{
    Liste *p ;
    P= (liste_db) malloc (sizeof (liste_db)) ;
    p->info=x ;
    If (liste==null)
    {
        p->suiv=null ;
        p->pre=null ;
        Liste=p ;
    }
    Else
    {
        p->suiv=liste ;
        p->pre=null ;
        Liste->pre=p ;
        Liste=p ;
    }
}
```

Listes doublement chaînées

Insertion au début

```
Cellule * newElement(Element *prev, Element *next, int e)
{
    Cellule *nouv;
    nouv = (Element*)malloc(sizeof(Element));
    if(nouv)
    {
        nouv->precedent=prev;
        nouv->suivant=next;
        nouv->info=e;
    }
    return nouv;
}
```

```
void errorDlist(char *s)
{
    printf(stderr, "%s", s);
    exit(EXIT_FAILURE);
}
```

Listes doublement chaînées

Les principales opérations sur
une liste doublement chaînée

1. Insertion
2. Affichage
3. Suppression

Listes doublement chaînées

```
void afficherApartirDebut(Dliste *L)
{
    Element *p;
    p=L->first;
    while(p)
    {
        printf("%i ",p->info);
        p=p->suivant;
    }
}
```

```
void afficherApartirFin(Dliste *L)
{
    Element *p;
    p=L->last;
    while(p)
    {
        printf("%i ",p->info);
        p=p->precedent;
    }
}
```

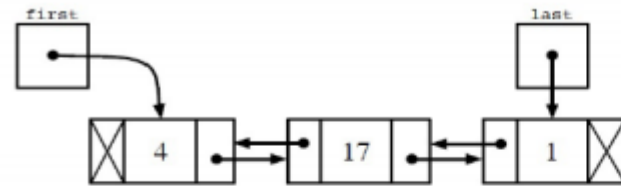
Listes doublement chaînées

Les principales opérations sur
une liste doublement chaînée

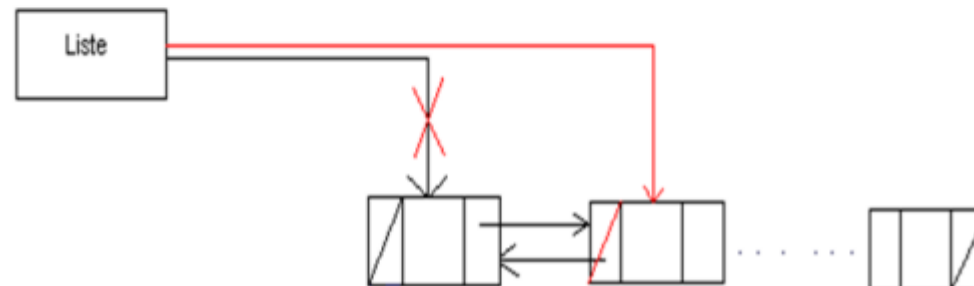
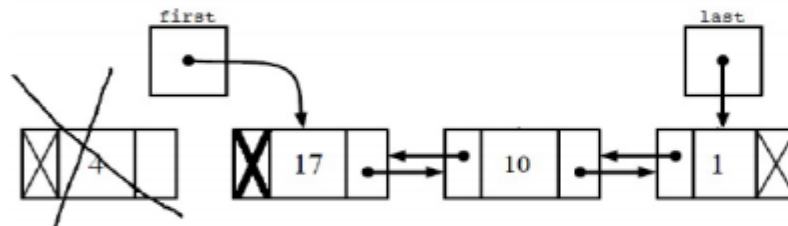
1. Insertion
2. Affichage
3. Suppression

Listes doublement chaînées

Supprimer le premier élément de la liste

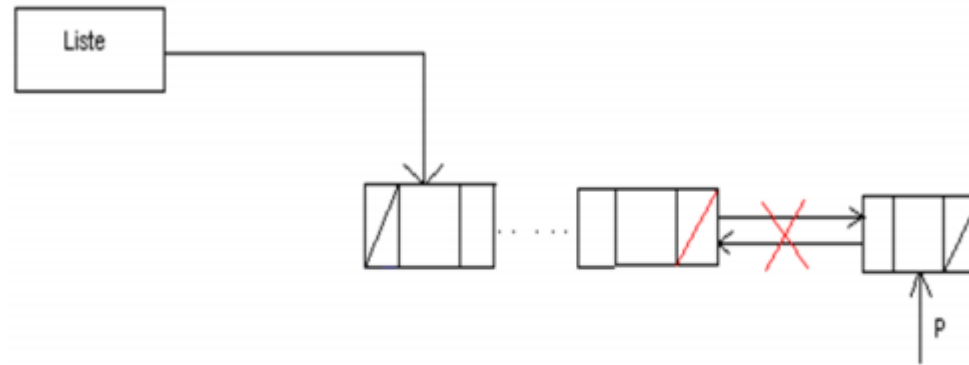


```
Void sup_debut (Dliste *liste)
{
    Liste = liste->suiv ;
    Liste->pre=null ;
}
```



Listes doublement chaînées

Supprimer le dernier élément de la liste



```
Void sup_fin (Dliste *liste)
{
    Dliste *p ;
    If (liste== null)
        Printf("liste vide") ;
    Else
        If(liste->suiv== null)
            Liste=null ;
        Else
```

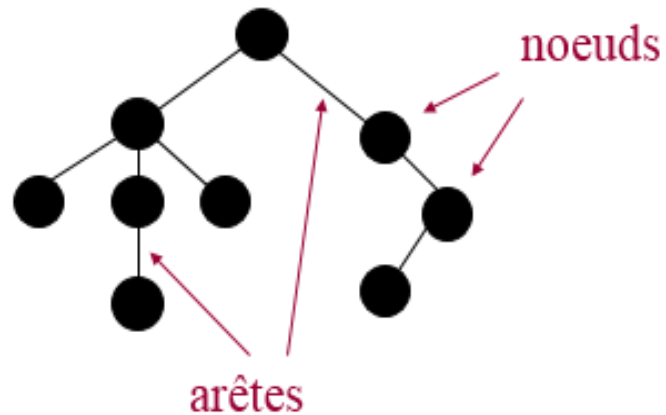
```
{
    P= (liste_db*) malloc (sizeof (liste_db)) ;
    P=liste ;
    While (p -> suiv !=null)
        P=p->suiv ;
    (p->pre)->suiv=null ;
}
```

Arbres

- Un **arbre** est une structure de données organisées de façon hiérarchique, à partir d'un nœud distingué appelé racine.
- Très importante en informatique!.
- Arbre de jeux (i.e., Echecs), système de fichiers UNIX/Windows, Arbres de tri etc.
- Nous étudierons deux types d'arbres : **Arbre Binaires de Recherches** et **Arbres équilibrés**

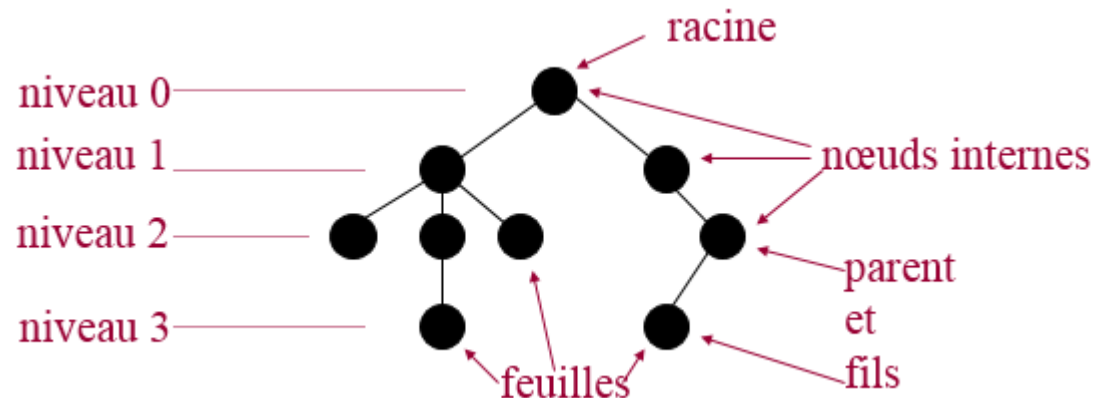
Arbres : définitions

- Un arbre est un ensemble de **Nœuds**, reliés par des **Arêtes**. Entre deux nœuds il existe toujours un seul chemin.



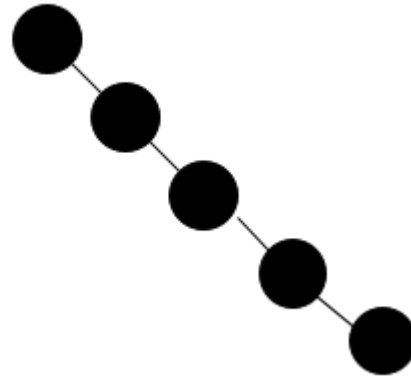
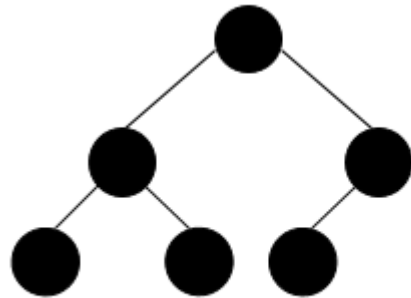
Arbres : définitions

- Les arbres sont enracinés. Une fois la **racine** définit tous les nœuds admettent un **niveau**.
- Les arbres ont des noeuds **internes** et des **feuilles** (nœuds externes). Chaque noeud (à l'exception de la racine) a un **parent** et admet zéro ou plusieurs **fil**s.



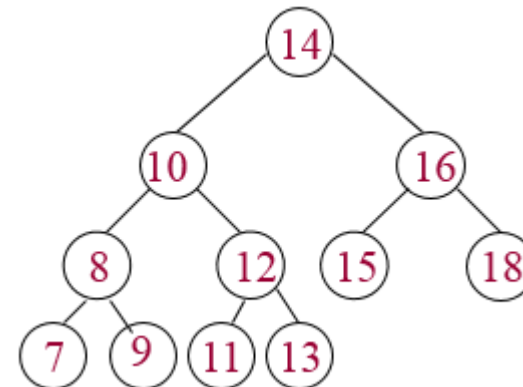
Arbres binaires

- Un **Arbre Binaire** est un arbre où chaque nœud admet au plus 2 fils.



Arbres binaires : définitions

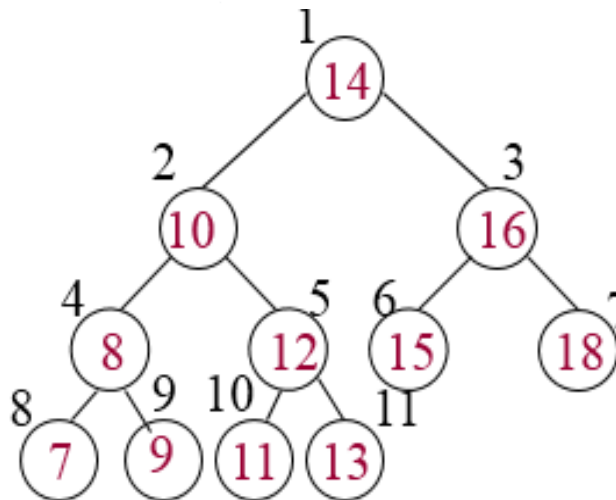
- Nœuds d'un arbre contiennent des **clés** (mots, nombres, etc)
- **Arbre Binaire parfait** : les feuilles sont toutes situées dans les deux derniers niveaux. Les feuilles du dernier niveau sont toutes à gauche.



Arbres binaires : représentation par tableaux

- Un arbre binaire complet peut être représenté par un tableau A avec un accès en $O(1)$ à chaque noeud:
 - Mémoriser les neouds séquentiellement de la racine aux feuilles et de gauche vers la droite.
 - Fils gauche de $A[i]$ est en $A[2i]$
 - Fils droit de $A[i]$ est en $A[2i + 1]$
 - Parent de $A[i]$ est en $A[i/2]$

Arbres binaires : représentation par tableaux



tab A:

1	2	3	4	5	6	7	8	9	10	11
14	10	16	8	12	15	18	7	9	11	13

Arbres binaires : représentation par pointeurs

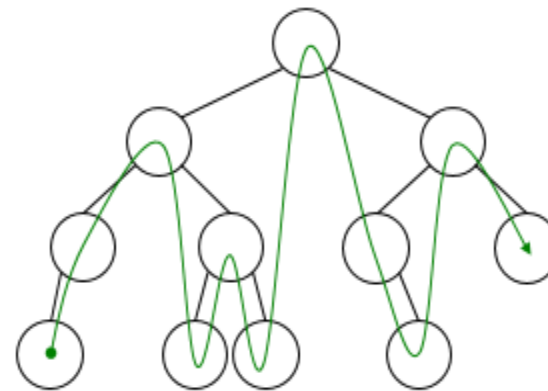
```
typedef struct n{  
    int clé;  
    struct n *fGauche, *fDroit;  
}nœud;
```

```
typedef nœud * Arbre;
```

Parcours : inOrdre

InOrdre est décrit réursivement :

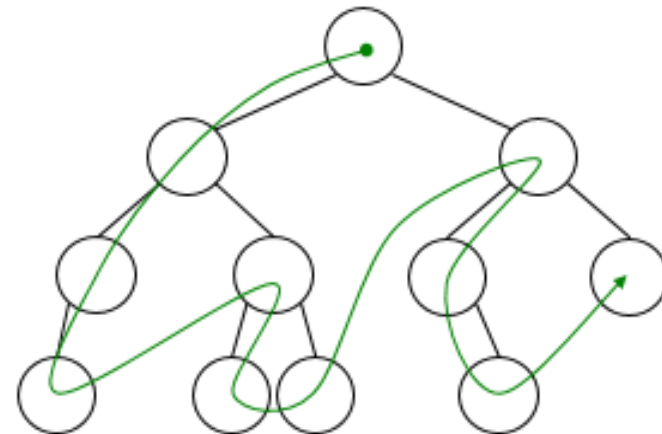
- Visiter le sous-arbre gauche en InOrdre
- **Visiter la racine**
- Visiter le sous-arbre droit en InOrdre



Parcours : PréOrdre

PréOrdre est décrit réursivement :

- **Visiter la racine**
- Visiter le sous-arbre gauche en PréOrdre
- Visiter le sous-arbre droit en PréOrdre



Parcours : non-récurusif

PréOrdre itératif en utilisant une Pile.

Pile S

empiler racine dans S

répéter jusqu'à $S = \emptyset$

v = dépiler S

si v $\neq$ nil

visiter v

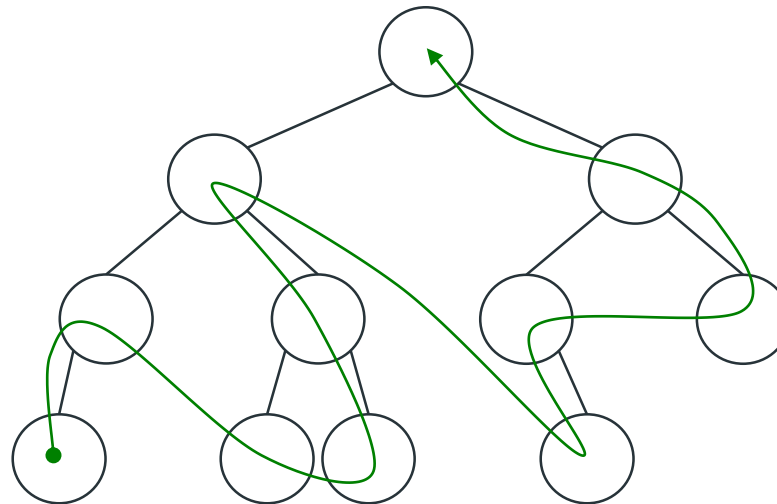
empiler le fils droit de v dans S

empiler le fils gauche de v dans S

Parcours : postOrdre

PostOrdre est décrit réursivement :

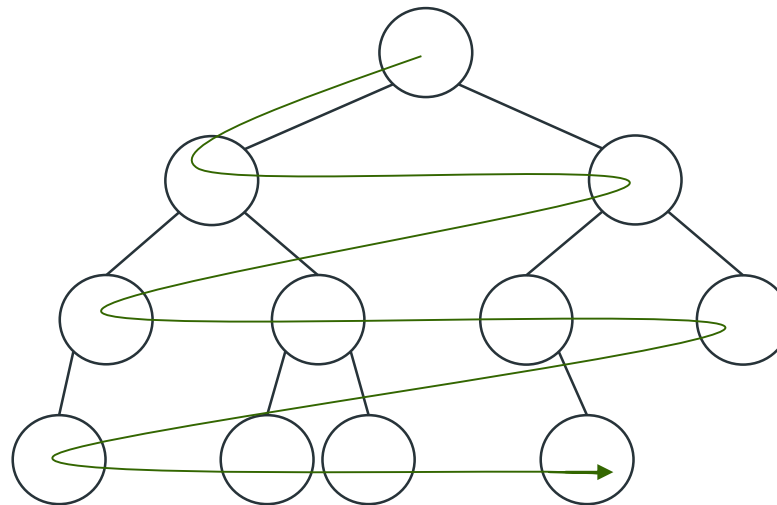
- Visiter le sous-arbre gauche en PostOrdre
- Visiter le sous-arbre droit en PostOrdre
- **Visiter la racine**



Parcours : levelOrdre

LevelOrdre visite les noeuds niveau par niveau depuis la racine:

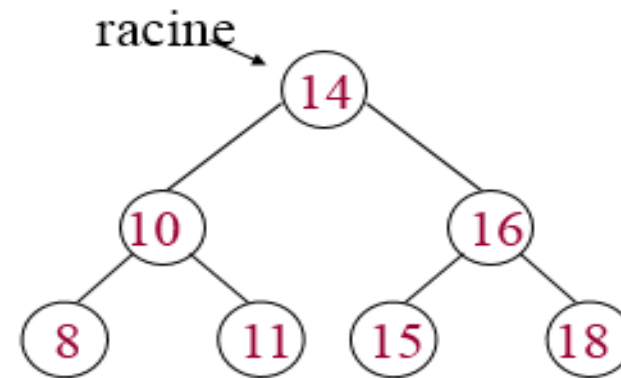
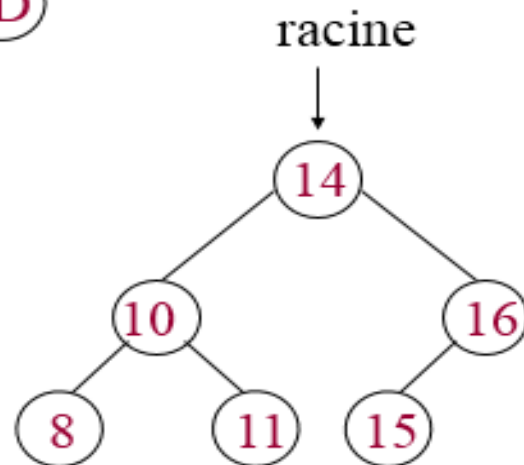
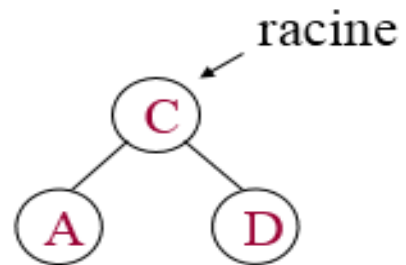
- Peut être décrit facilement en utilisant une File (**Comment??**)
- Parcours appelé “Breadth First Search” (parcours en largeur) dans les graphes



Arbre Binaire de Recherche

- Un **Arbre Binaire de Recherche (ABR)** est un arbre binaire avec les propriétés suivantes :
 - La clé associée à un noeud est *supérieur* aux clés des nœuds de son sous-arbre gauche
 - La clé associée à un noeud est *inférieur* aux clés des nœuds de son sous-arbre droit

Arbre Binaire de Recherche: Exemples



Arbre Binaire de Recherche

- ABR est un arbre avec la **propriété** suivante :

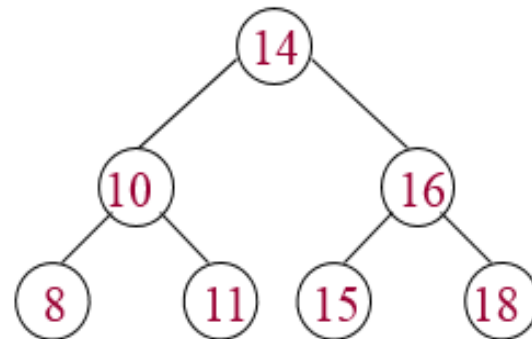
Clé.fGauche < Clé.parent < Clé.fDroit

NOTER! Le parcours InOrdre visite les clés dans l'ordre croissant.

```
void inOrdre(Arbre racine) {  
    inOrdre(racine->fGauche)  
    print(racine->key)  
    inOrdre(racine->fDroit)  
}
```

ABR: InOrdre

Exemple:

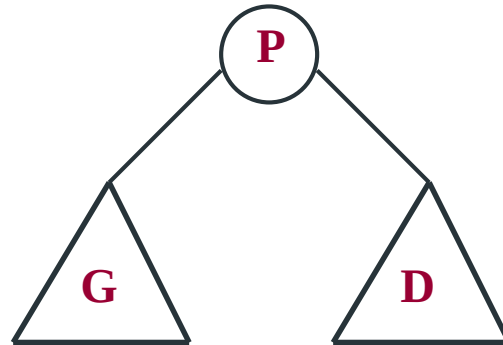


InOrdre visites :

(8)
(10)
(11)
(14)
(15)
(16)
(18)

ABR : Rechercher un élément

Soit un ABR :



Problème: rechercher un noeud avec une clé x ?

ABR : Rechercher un élément

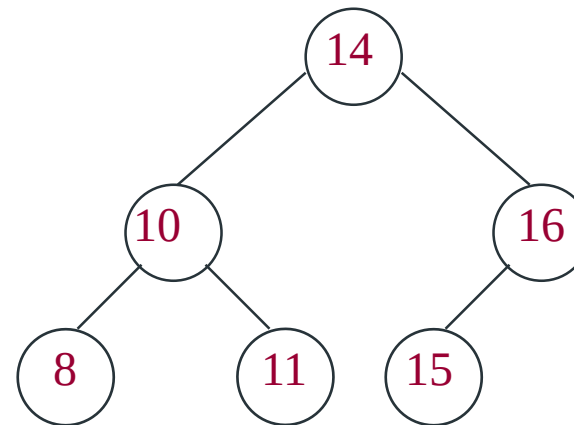
rechercher(racine, x)

comparer x à la clé de racine:

- si $x = \text{clé}$ return
- si $x < \text{clé} \Rightarrow$ chercher dans G
- si $x > \text{clé} \Rightarrow$ chercher dans D

**chercher de la même manière
dans G ou D**

Exemple:



x=8 (oui) x=17 (non)

ABR : Rechercher un élément

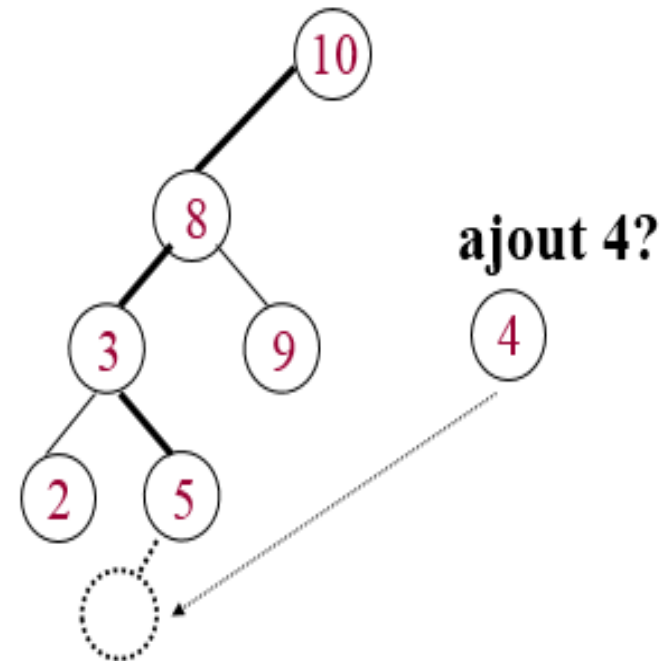
```
bool searchABR(Arbre racine; typeCle clé){  
    if (racine==NULL)    return false  
    if (racine->clé==clé)  
        return true;  
    else if (key < racine->clé)  
        return searchABR(racine->fGauche, clé);  
    else  
        return searchABR(racine->fDroit, clé)  
}
```

ABR : Ajout d'un élément

Comment ajouter une clé?

La même procédure que searchABR s'applique:
Déterminer la position d'insertion par searchABR.
Ajouter la nouvelle clé si la recherche échoue.

Exemple:



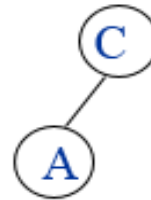
Construction d'un ABR

Exemple: ajouter **C A B L M** (dans l'ordre!)

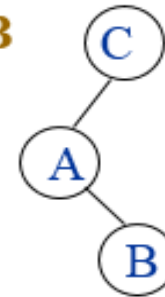
1) Ajouter C



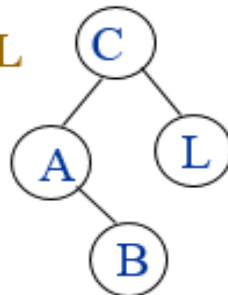
2) ajouter A



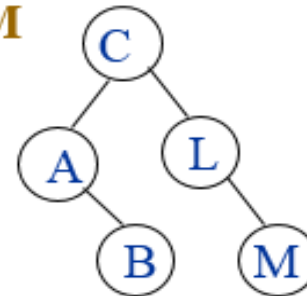
3) ajouter B



4) Ajouter L



5) Ajouter M



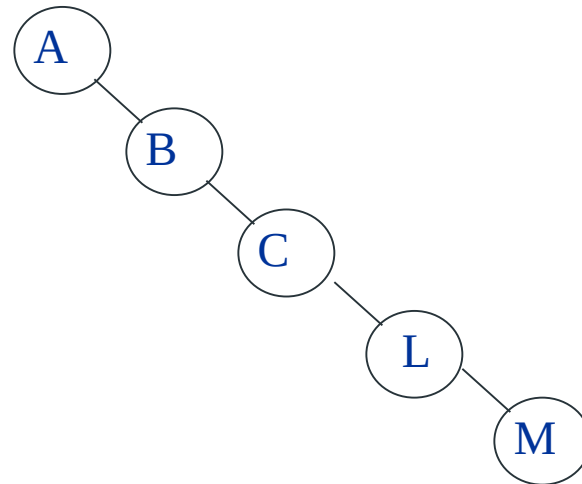
Construction d'un ABR

L'ABR est-il unique pour une séquence de lettres A
B C L M ?

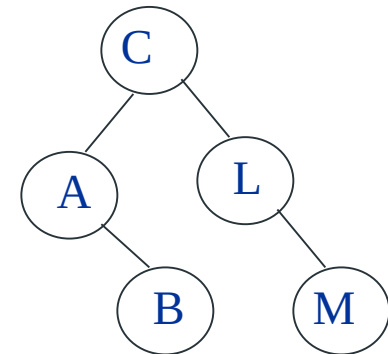
NON! différentes séquences donnent différents

ABR

Ajout de : **A B C L M**



Ajout de : **C A B L M**



Trier avec un ABR

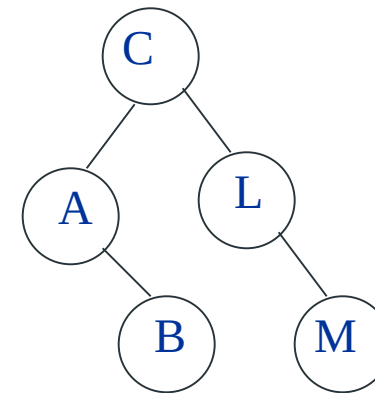
Soit un ABR, peut-on afficher les clés dans l'ordre?

Visiter l'ABR avec un parcours InOrdre:

- visiter le sous-arbre gauche
- afficher racine
- visiter le sous-arbre droit

Comment trouver le minimum?
Comment trouver le maximum?

Example:



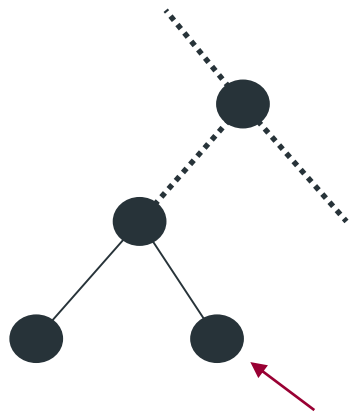
InOrdre affichage:

A B C L M

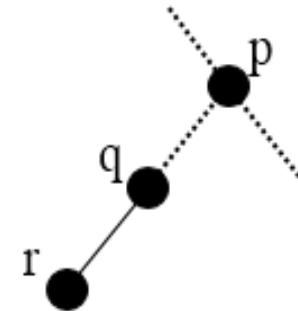
ABR : supprimer un élément

Pour supprimer un nœud contenant x ,
rechercher x , une fois trouvé appliquer l'un des trois
cas suivants:

CAS A: x est une feuille



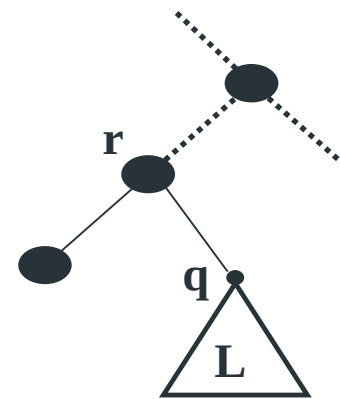
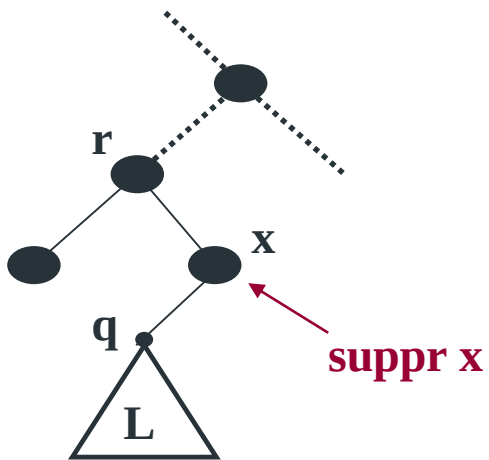
supprimer x



On obtient un ABR

ABR : supprimer un élément

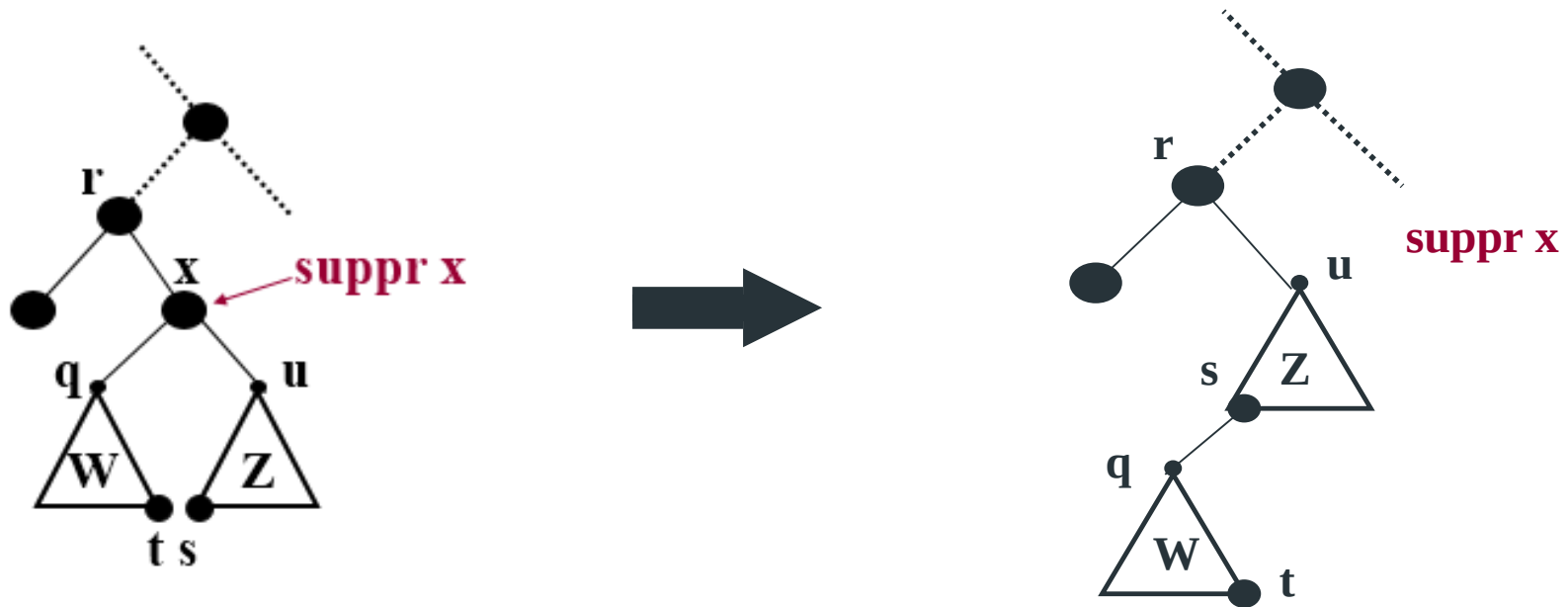
Cas B: x est un nœud interne avec un seul sous-arbre



On obtient un ABR

ABR : supprimer un élément

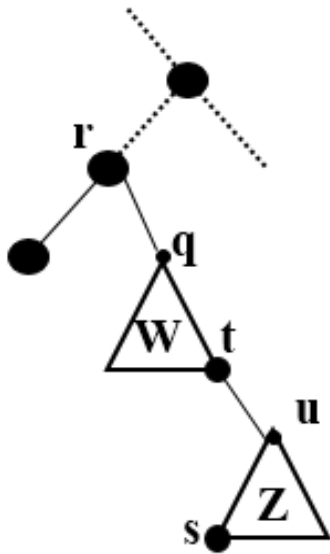
Cas C: x est un nœud interne avec 2 sous-arbres



propriété ABR est conservé

ABR : supprimer un élément

Cas C suite: ... ou encore comme suit



$$q < x < u$$

$\Rightarrow$ q est inférieur au plus petit élément de Z

$\Rightarrow$ r est supérieur au plus grand élément de W